Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

THM - L2 MAC Flooding & ARP Spoofing

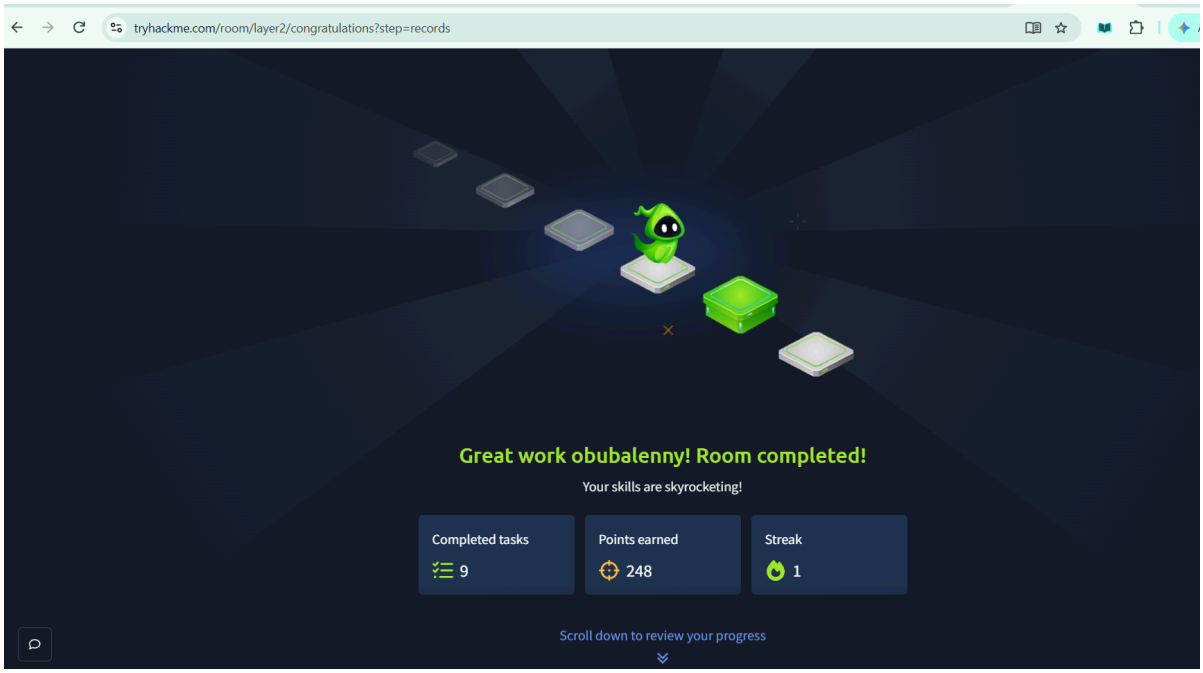
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 9, Assignment 2: THM - L2 MAC Flooding & ARP Spoofing*
Platform	*TryHackMe*
Date submitted	*08/11/2026*

1. Module completion

Shareable link

https://tryhackme.com/room/layer2/congratulations?step=records

Completion screenshot



2. Introduction

This module covers Layer 2 attacks, MAC flooding to overwhelm a switch's CAM table and force it into broadcasting traffic to every port (defeating its normal traffic isolation), and ARP cache poisoning to trick hosts into routing their traffic through an attacker's machine for a man-in-the-middle position. This matters to a SOC analyst because these are attacks that happen entirely inside the local network, often invisible to perimeter-focused monitoring, and understanding them is what lets an analyst recognize abnormal ARP traffic or switch behavior as a sign of lateral movement rather than noise. Going into this room, I expected to build on the networking fundamentals I had learned from switches, MAC tables, ARP , and see concretely how trusting those protocols by design becomes an exploitable weakness.

3. The work

3.1 Initial Access

What this section covered

This section set up the scenario of having already gained initial access and root on a Linux host during a pentest, and discovering it's dual-homed (connected to two networks), creating an opportunity to pivot laterally into a second network segment. It had me reconnect via SSH using provided credentials to simulate picking the engagement back up through established persistence.

Question

Now, can you (re)gain access? (Yay/Nay)

Answer

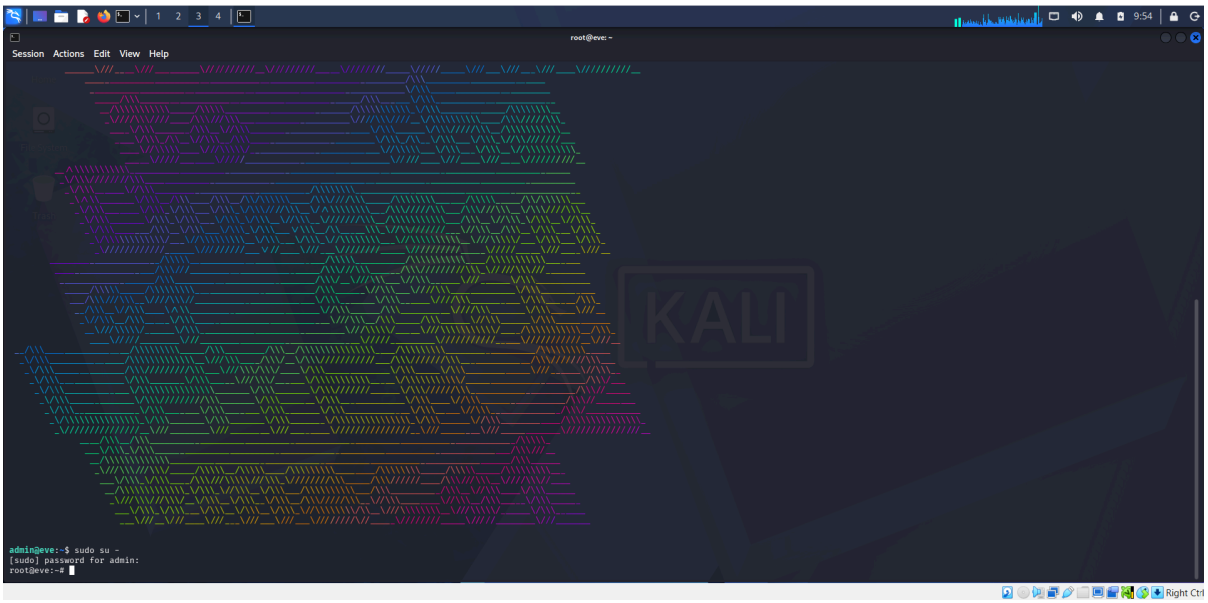
Yay

How you got there

I connected using the command given on the room page - ssh -o StrictHostKeyChecking=accept-new admin@10.49.137.24

Evidence

Shows the terminal output of the SSH login succeeding and the sudo su - escalation to a root prompt



Confirms access to the compromised host was successfully (re)established with root privileges.

3.2 Network Discovery

What this section covered

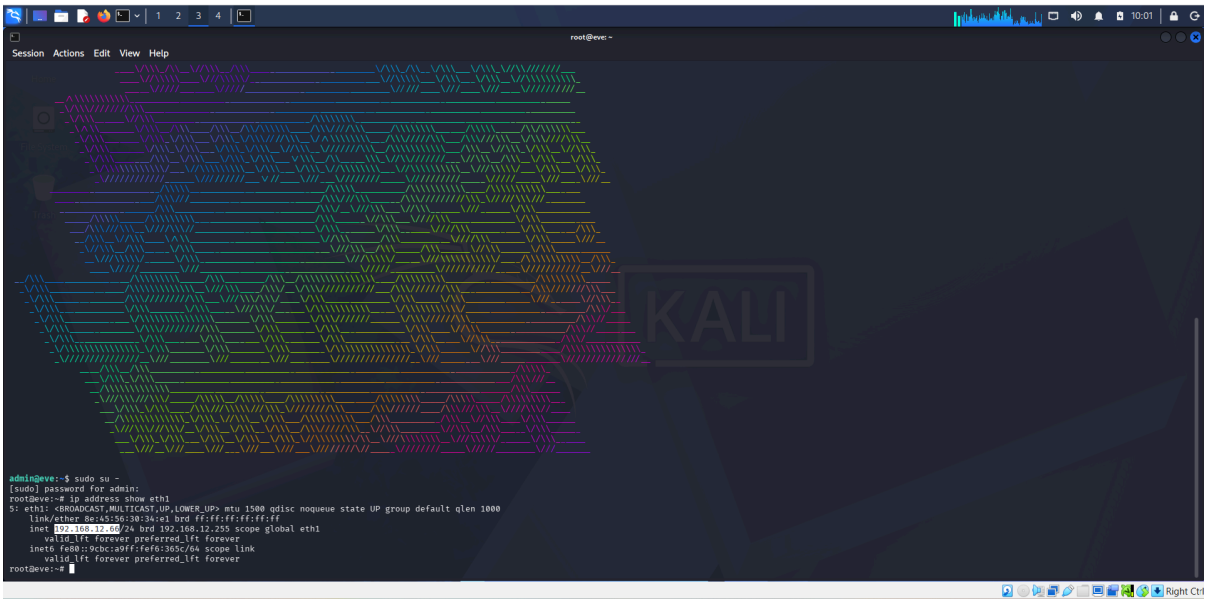
This section had me enumerate the second network attached to the dual-homed host (eth1), first reading my own IP and subnet off that interface, then using nmap to sweep the subnet and discover other live hosts I could potentially pivot to.

Question 1

What is your IP address?

Answer

192.168.12.66

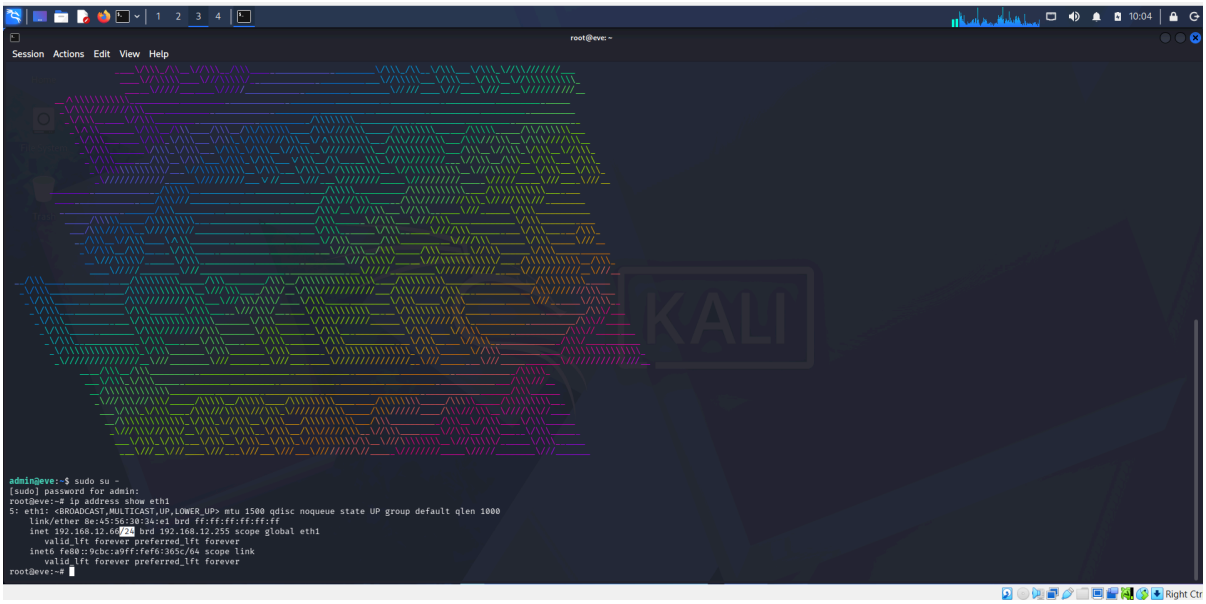


Question 2

What's the network's CIDR prefix?

Answer

/24



Question 3

How many other live hosts are there?

Answer

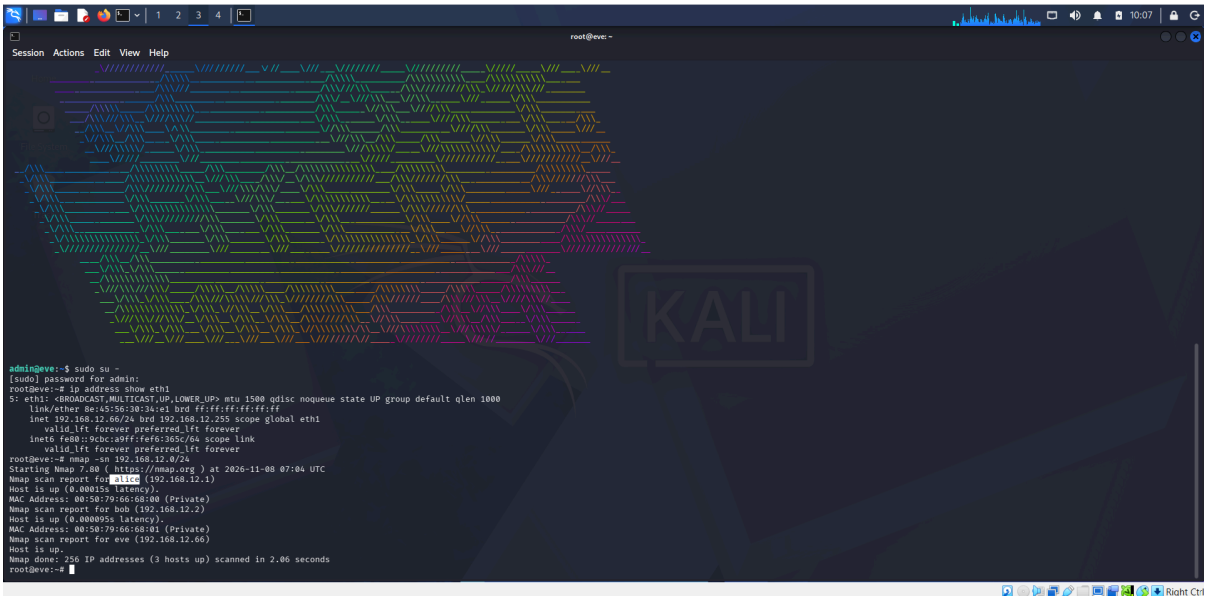
2

Question 4

What's the hostname of the first host (lowest IP address) you've found?

Answer

alice



How you got there

I ran ip address show eth1 and found the inet line showing 192.168.12.66/24 on the eth1 adapter, giving me both my IP and the subnet's CIDR prefix. From that I worked out the network as 192.168.12.0/24. I then ran a ping sweep across the full subnet with:

nmap -sn 192.168.12.0/24

which scanned all 256 addresses and reported 3 hosts up: eve (192.168.12.66, myself), alice (192.168.12.1), and bob (192.168.12.2). Excluding myself, that's 2 other live hosts, with alice at the lowest IP address.

Evidence

Shows the terminal output of the nmap -sn sweep listing the three live hosts with their hostnames, IPs, and MAC addresses.



This confirms the second network segment was successfully enumerated and two additional pivot targets (alice, bob) were identified by hostname and IP.

3.3 Passive Network Sniffing

What this section covered

This section had me passively sniff traffic on the eth1 interface with tcpdump to see what, if anything, I could learn about activity between alice and bob without actively scanning them, demonstrating that even without port-scanning, a host sitting on the same network segment can observe traffic flowing past it.

Question 1

Can you see any traffic from those hosts? (Yay/Nay)

Answer

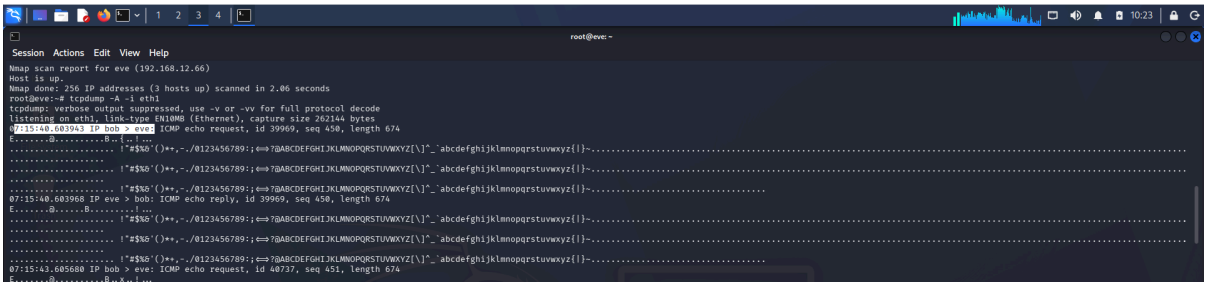
Yay

Question 2

Who keeps sending packets to eve?

Answer

bob

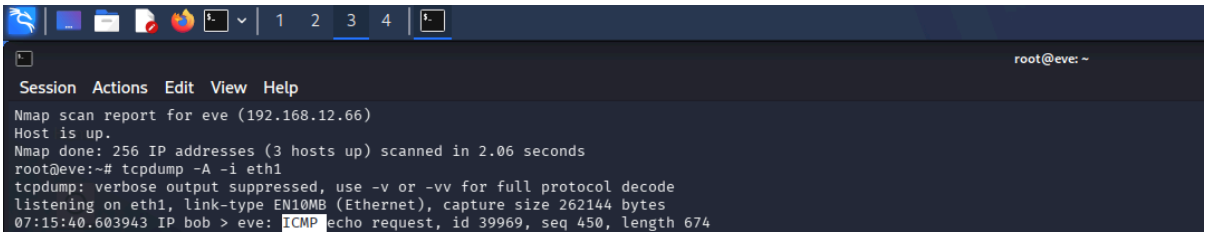


Question 3

What type of packets are sent?

Answer

ICMP



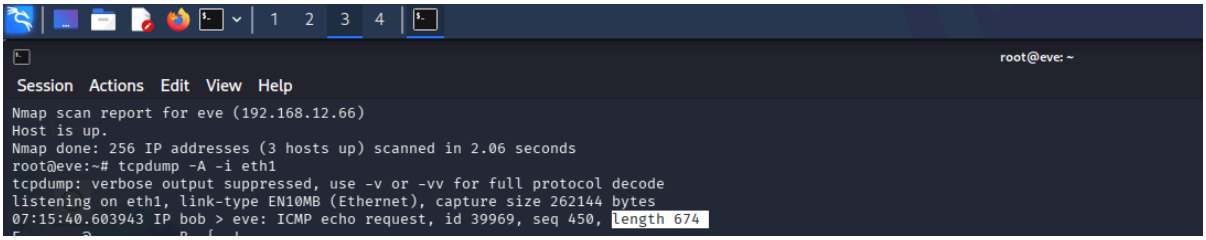
Question 4

What's the size of their data section? (bytes)

Answer

666

Explanation - tcpdump's length 674 for ICMP packets refers to the total ICMP message size (8-byte ICMP header + data). So data section = 674 − 8 = 666 bytes.

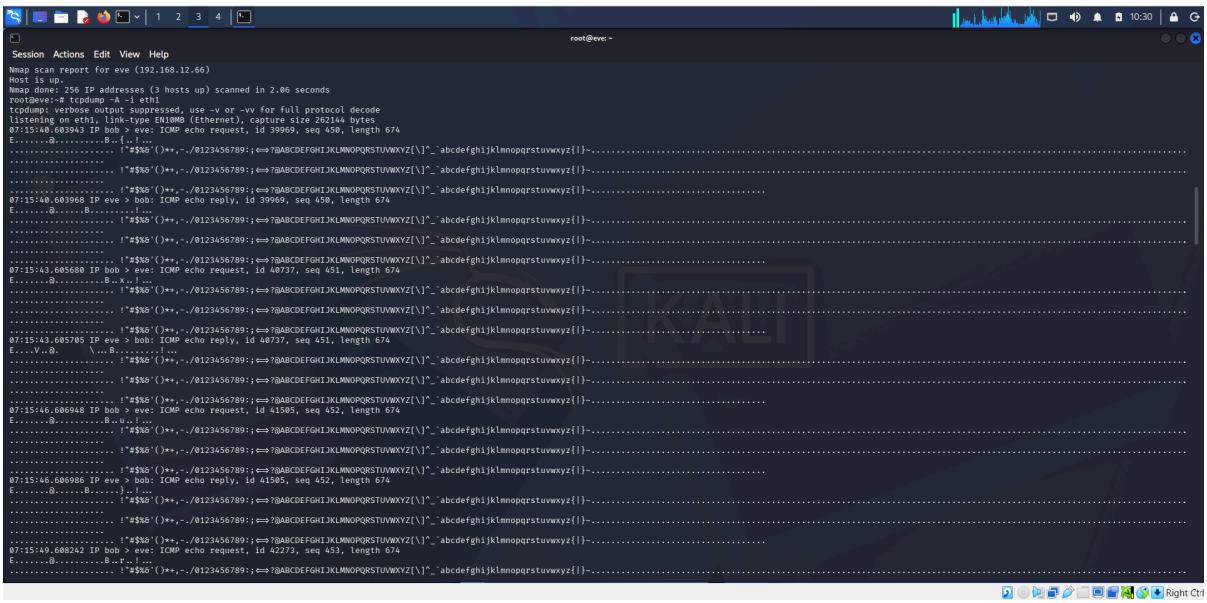


How you got there

I ran tcpdump -A -i eth1 and immediately saw a steady stream of ICMP echo request/reply pairs between bob and eve, roughly every 3 seconds, along with two ARP request/reply exchanges where eve resolved bob's MAC address. Each ICMP packet was reported with length 674; since tcpdump's reported length for ICMP includes the 8-byte ICMP header, the data section itself is 674 − 8 = 666 bytes. The live capture answered all four questions directly, so I didn't need to write to a pcap file or inspect it in Wireshark for this task.

Evidence

The terminal output of tcpdump -A -i eth1 showing the repeating ICMP echo request/reply exchange between bob and eve, and the ARP resolution for bob.



This proves that passive sniffing on a switched network segment can reveal live traffic patterns (here, bob continuously pinging eve) without needing to actively probe any host.

3.4 Sniffing while Mac Flooding

What this section covered

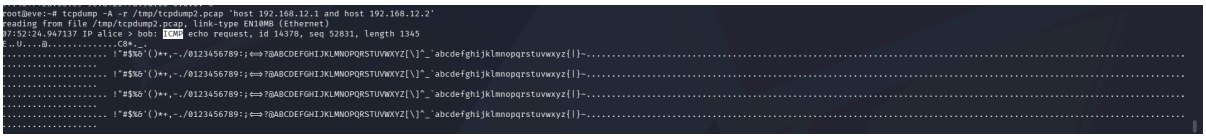
This section demonstrated the actual impact of MAC flooding: exhausting the switch's CAM table with macof so it fails open and starts behaving like a hub, broadcasting every frame to every port instead of only to the intended destination. That made traffic between alice and bob visible to eve.

Question 1

What kind of packets is Alice continuously sending to Bob?

Answer

ICMP



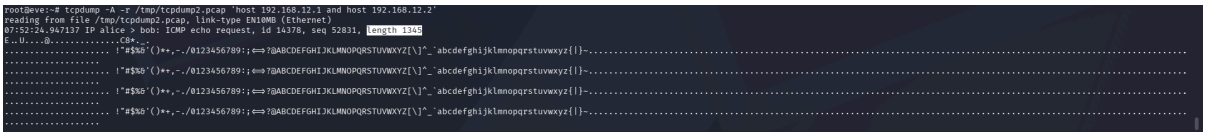
Question 2

What's the size of their data section? (bytes)

Answer

1337

Explanation - 1345 − 8 = 1337 bytes (the ICMP header is 8 bytes).



How you got there

I opened a second SSH session to run tcpdump -A -i eth1 -w /tmp/tcpdump2.pcap in the first session while launching macof -i eth1 in the second. After about 30 seconds I stopped both; the

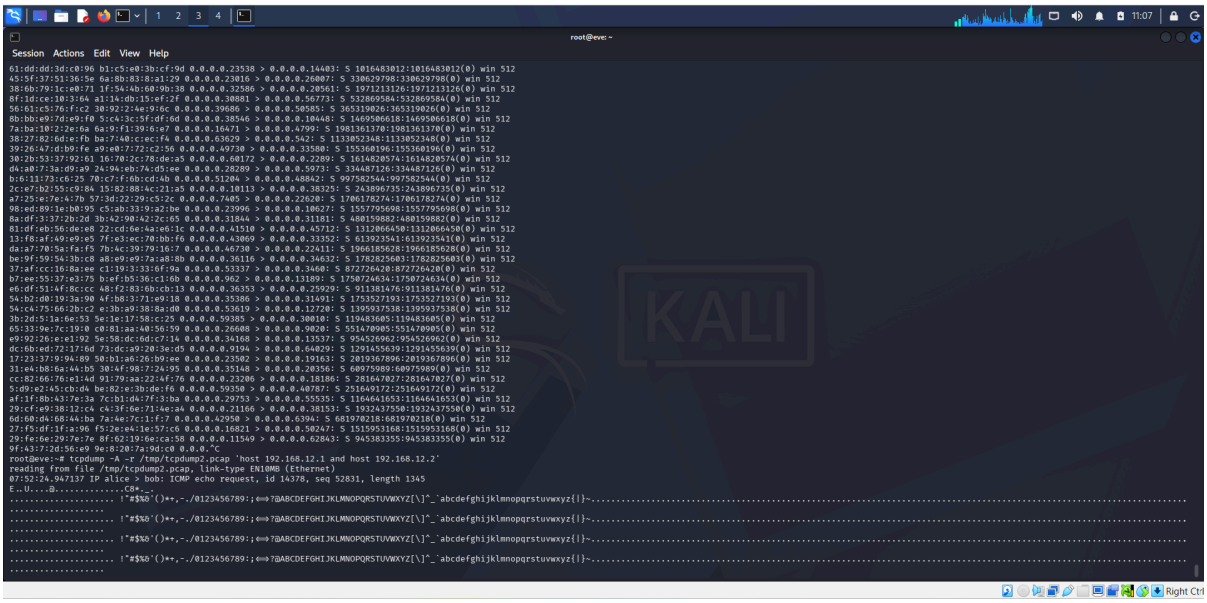
capture had grown to 272,580 packets, almost entirely the random spoofed SYN traffic macof itself generates while flooding the switch's MAC table. Rather than transferring that large file to Wireshark, I filtered it directly on the host by replaying the capture against just the two hosts of interest:

tcpdump -A -r /tmp/tcpdump2.pcap 'host 192.168.12.1 and host 192.168.12.2'

This isolated genuine alice and bob traffic from the flood noise and revealed a steady stream of ICMP echo requests from alice to bob, each reported with length 1345 and subtracting the 8-byte ICMP header gives a data section of 1337 bytes.

Evidence

Shows the terminal output of the filtered tcpdump replay showing repeated alice > bob: ICMP echo request ... length 1345 entries.



Confirms that MAC flooding successfully forced the switch into fail-open mode, exposing traffic between two hosts that shouldn't have been visible to a third party on a properly functioning switched network.

3.4 Man in the middle: Intro to ARP Spoofing

What this section covered

This section introduced ARP cache poisoning as a quieter alternative to MAC flooding, rather than brute-forcing the switch, the attacker sends forged ARP replies to trick two hosts into associating the attacker's MAC address with each other's IP, redirecting their traffic through the attacker transparently. It also demonstrated that this attack isn't guaranteed to work: hosts that validate incoming ARP replies (checking them against requests they actually sent, for example) can detect and reject the forged packets.

Question 1

Can ettercap establish a MITM in between Alice and Bob? (Yay/Nay)

Answer

Nay

Question 2

Would you expect a different result when attacking hosts without ARP packet validation enabled? (Yay/Nay)

Answer

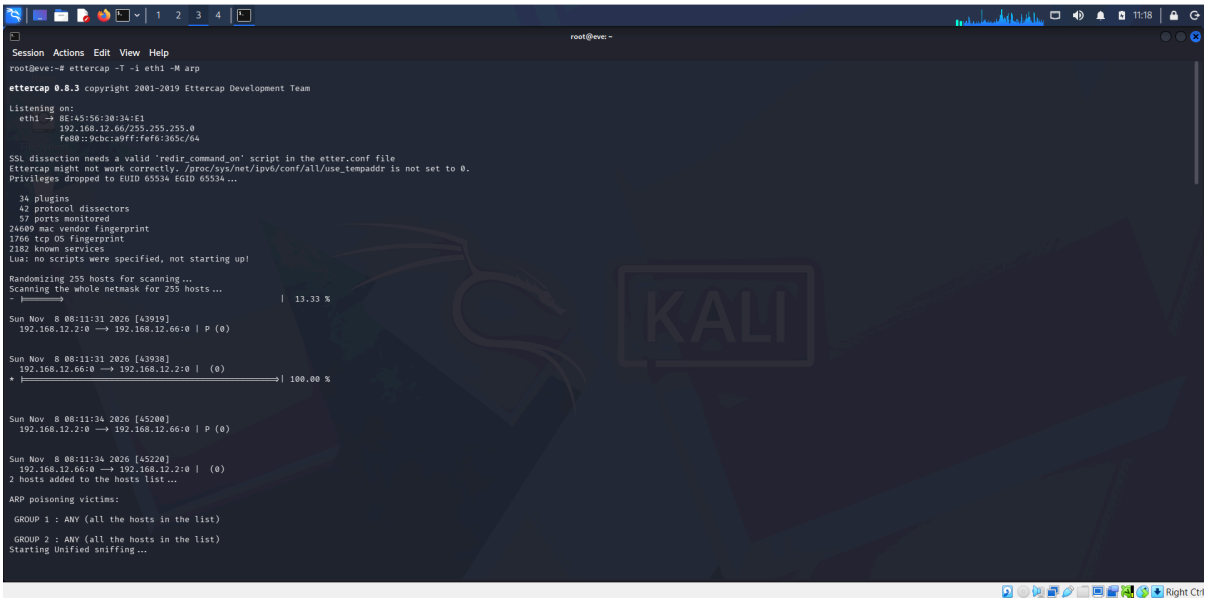
Yay

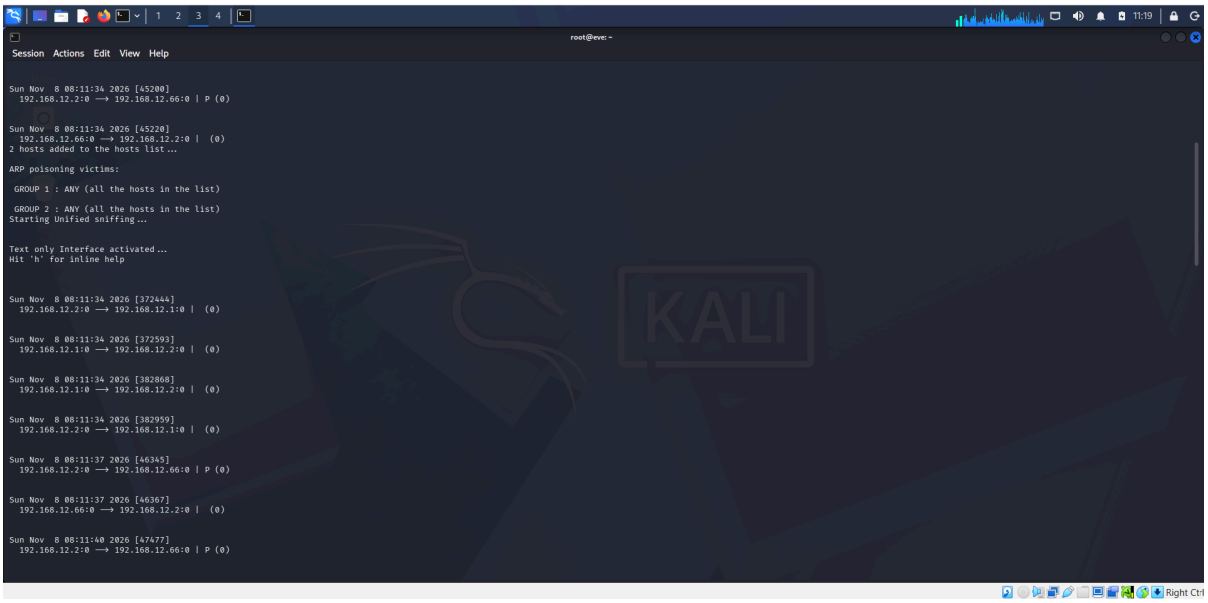
How you got there

I ran ettercap -T -i eth1 -M arp on eve, which scanned the subnet, found alice and bob, and set both as poisoning targets ("GROUP 1: ANY" / "GROUP 2: ANY"). Over the ~50 seconds it ran, the only traffic ettercap reported was between bob (192.168.12.2) and eve (192.168.12.66) itself, the normal traffic eve is already a legitimate party to. No alice and bob traffic appeared rerouted through eve, meaning the poisoning attempt did not succeed in inserting eve into their communication. This matches the room's framing: since both hosts validate incoming ARP replies, they rejected eve's forged ARP announcements rather than updating their ARP caches with the spoofed mapping.

Evidence

Shows the terminal output of ettercap's ARP poisoning attempt, showing it targeting both hosts but only ever capturing eve's own legitimate traffic with bob.





Confirms ARP packet validation on alice and bob successfully defeated the MITM attempt, in contrast to what would happen against unprotected hosts.

3.5 Man in the middle - Sniffing

What this section covered

> *Two or three sentences in your own words. What was the idea, and what did you take from it.*

[your summary here]

Question 1

Scan the network on eth1. Who's there? Enter their IP addresses in ascending order.

Answer

192.168.12.10, 192.168.12.20

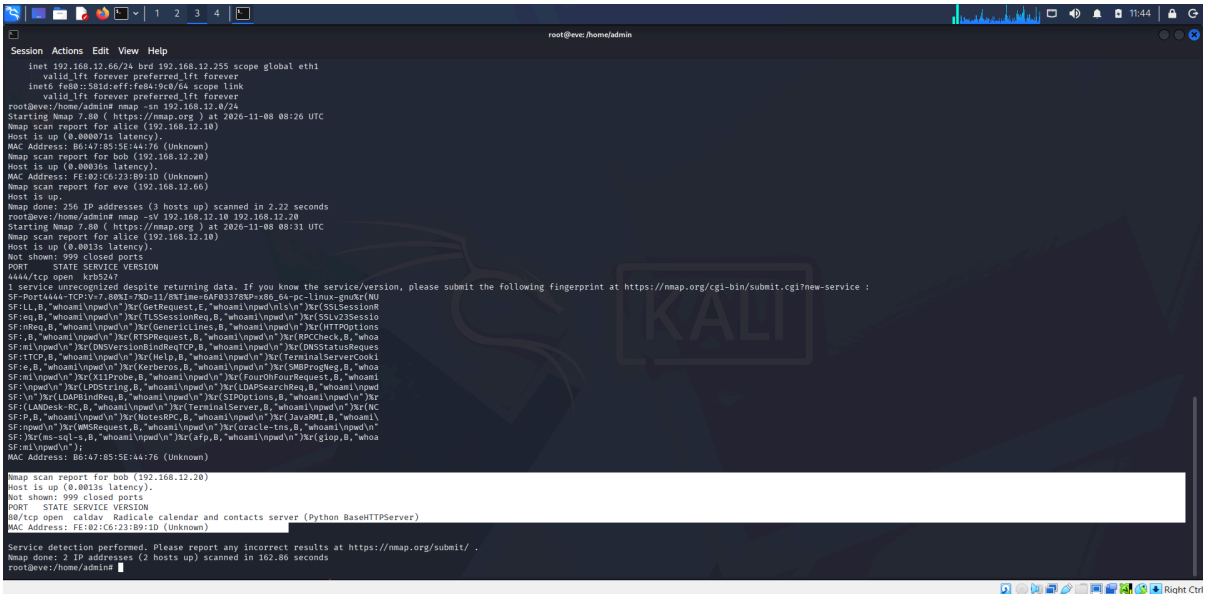


Question 2

Which machine has an open well-known port?

Answer

192.168.12.10, 192.168.12.20



Question 3

What is the port number?

Answer

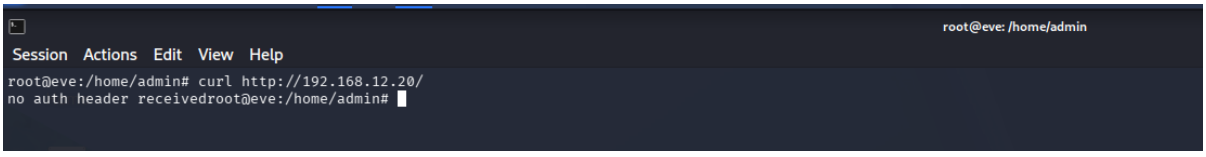
80

Question 4

Can you access the content behind the service from your current position? (Nay/Yay)

Answer

Nay

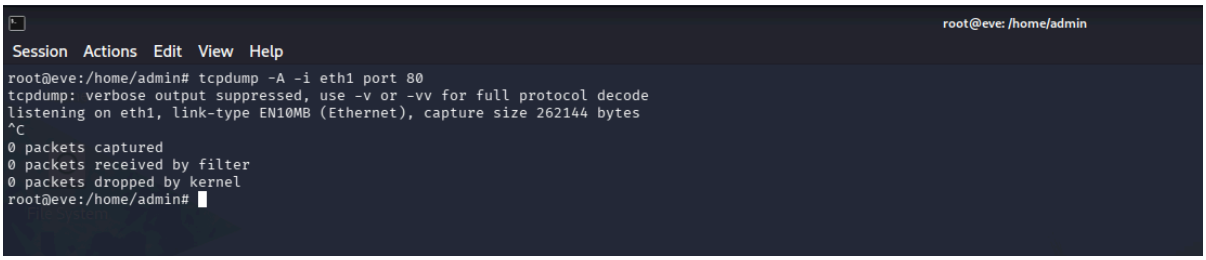


Question 5

Can you see any meaningful traffic to or from that port passively sniffing on you interface eth1? (Nay/Yay)

Answer

Nay

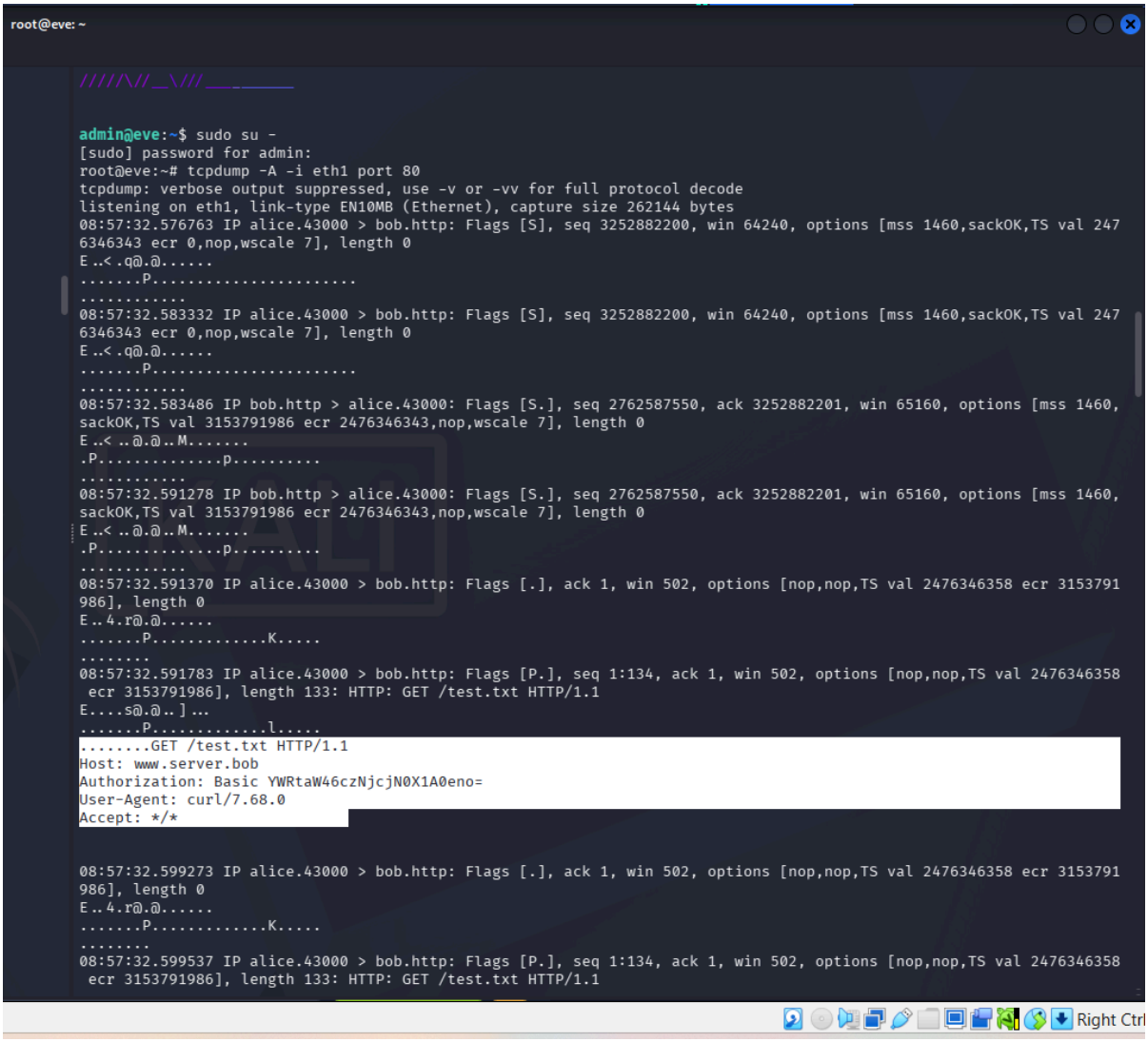


Question 6

Now launch the same ARP spoofing attack as in the previous task. Can you see some interesting traffic, now? (Nay/Yay)

Answer

Yay



Question 7

Who is using that service?

Answer

alice

```
admin@eve:~$ sudo su -
[sudo] password for admin:
root@eve:~# tcpdump -A -i eth1 port 80
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
08:57:32.576763 IP alice.43000 > bob.http: Flags [S], seq 3252882200, win 64240, options [mss 1460,sackOK,TS val 247
6346343 ecr 0,nop,wscale 7], length 0
E ..< .q@.@......
```

Question 8

What's the hostname the requests are sent to?

Answer

www.server.bob

```
.......
08:57:32.591783 IP alice.43000 > bob.http: Flags [P.], seq 1:134, ack 1, win 502, options [nop,nop,TS val 2476346358
 ecr 3153791986], length 133: HTTP: GET /test.txt HTTP/1.1
E....s@.@.. ] ...
.......P.............l.....
........GET /test.txt HTTP/1.1
Host: www.server.bob
Authorization: Basic YWRtaW46czNjcjN0X1A0eno=
User-Agent: curl/7.68.0
Accept: */*
```

Question 9

Which file is being requested?

Answer

test.txt

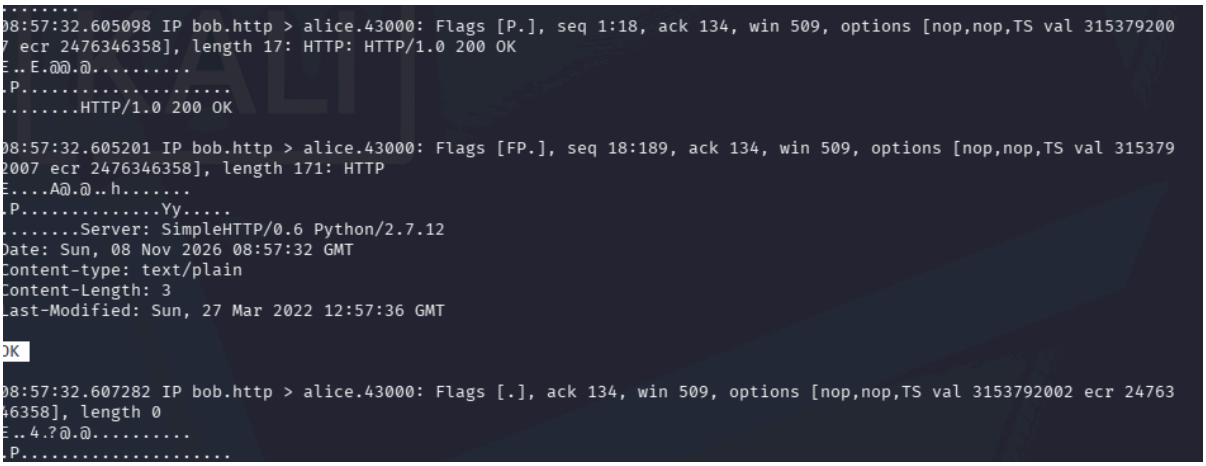
```
.......
08:57:32.599537 IP alice.43000 > bob.http: Flags [P.], seq 1:134, ack 1, win 502, options [nop,nop,TS val 2476346358
 ecr 3153791986], length 133: HTTP: GET /test.txt HTTP/1.1
E....s@.@.. ] ...
.......P.............l.....
........GET /test.txt HTTP/1.1
Host: www.server.bob
Authorization: Basic YWRtaW46czNjcjN0X1A0eno=
User-Agent: curl/7.68.0
Accept: */*
```

Question 10

What text is in the file?

Answer

OK



Question 12

Which credentials are being used for authentication? (username:password)

Answer

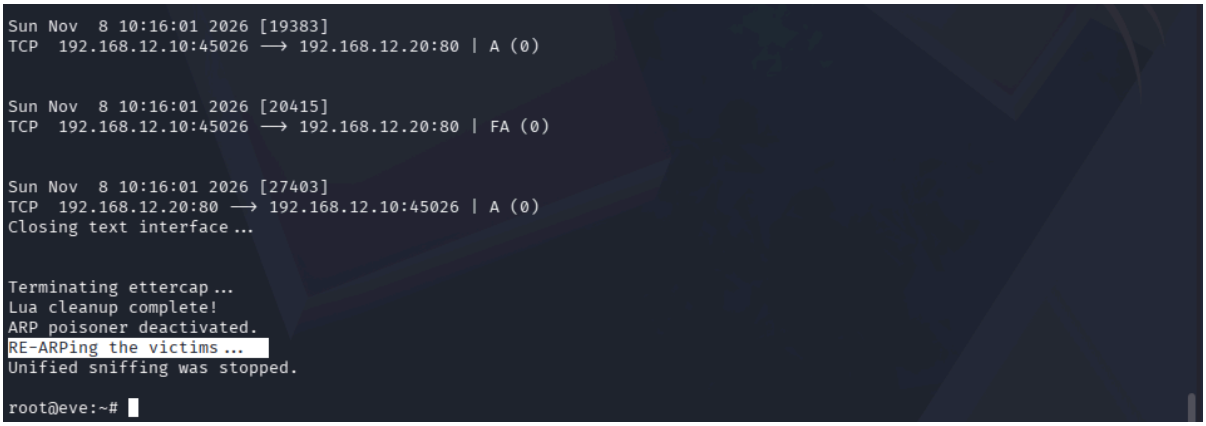
admin:s3cr3t_P4zz

Question 13

Now, stop the attack (by pressing q). What is ettercap doing in order to leave its man-in-the-middle position gracefully and undo the poisoning?

Answer

RE-ARPing the victims



Question 14

Can you access the content behind that service, now, using the obtained credentials? (Nay/Yay)

Answer

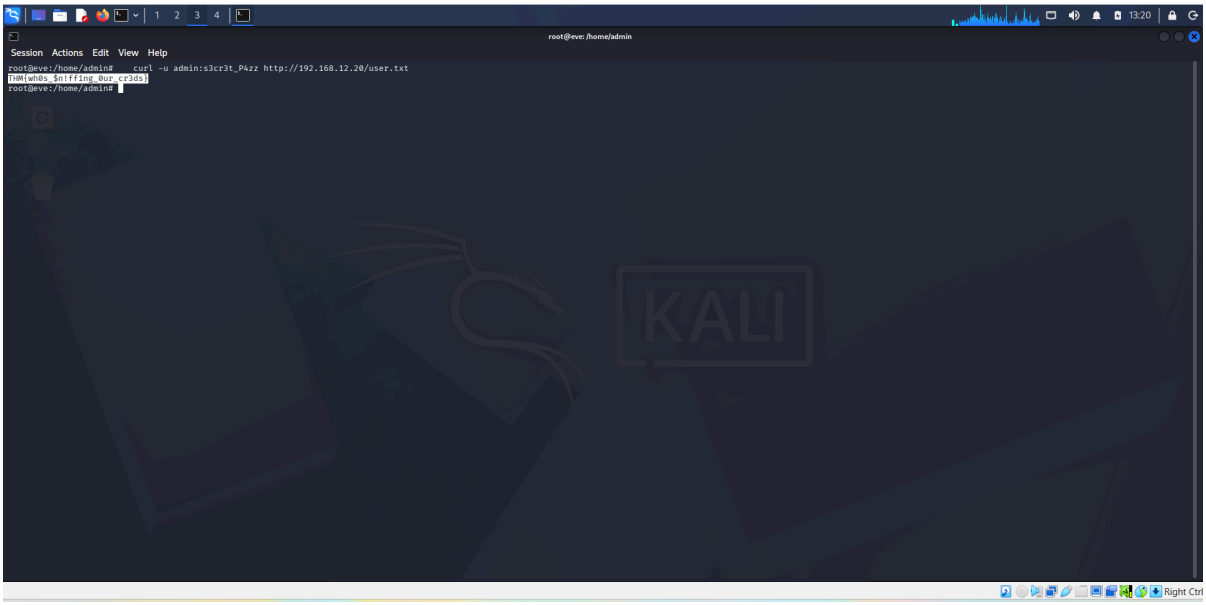
Yay

Question 15

What is the user.txt flag?

Answer

THM{wh0s_$n!ff1ng_0ur_cr3ds}

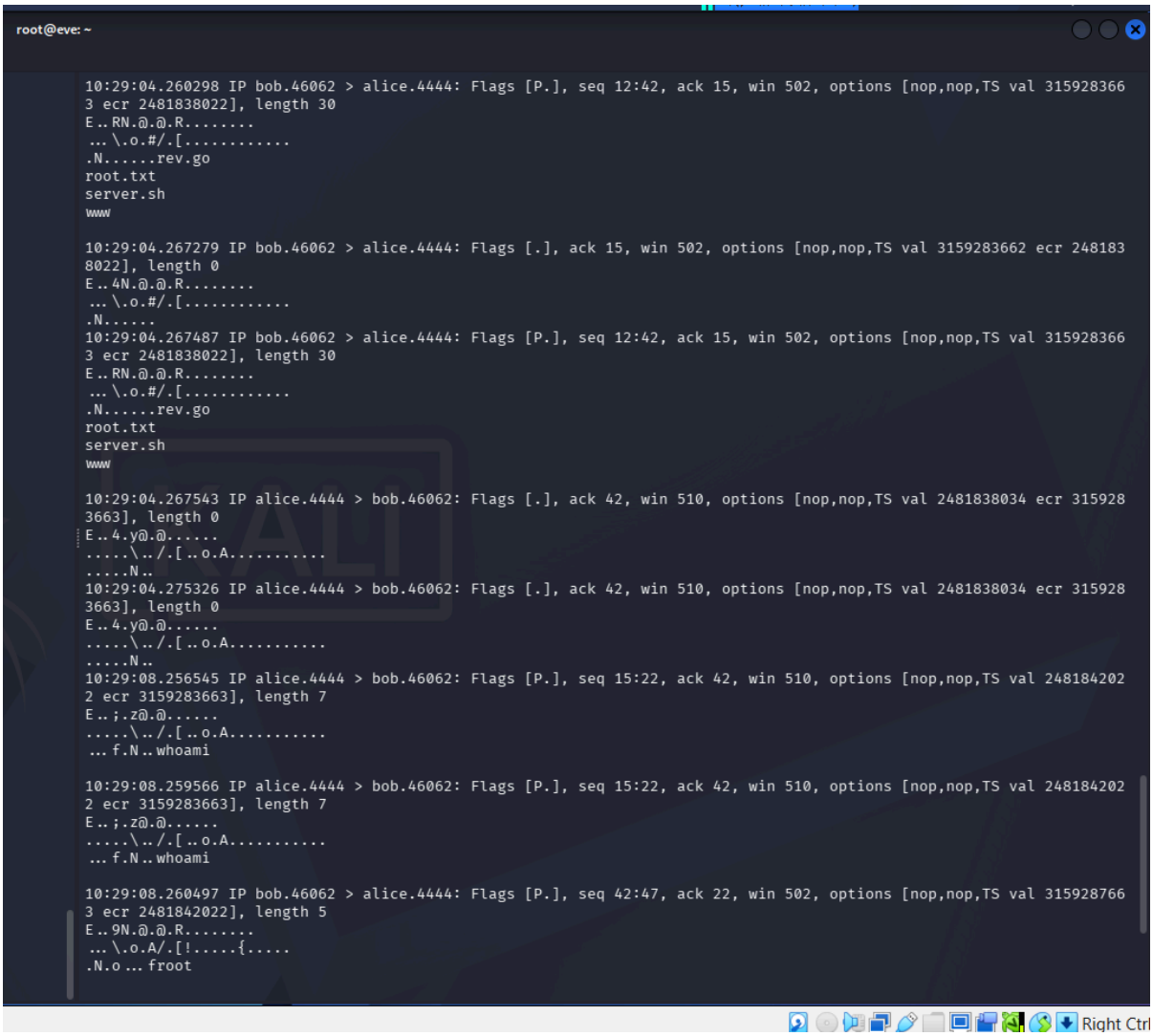


Question 16

You should also have seen some rather questionable kind of traffic. What kind of remote access (shell) does Alice have on the server?

Answer

Reverse shell



Question 17

What commands are being executed? Answer in the order they are being executed.

Answer

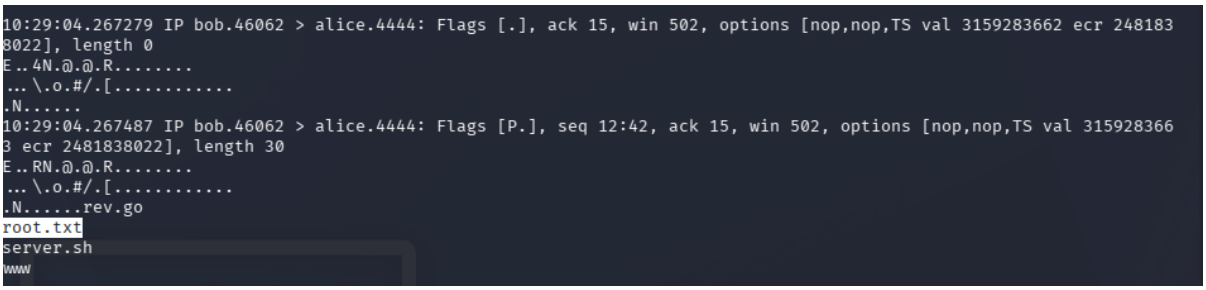
whoami, pwd, ls

Question 18

Which of the listed files do you want?

Answer

192.168.12.10, 192.168.12.20



How you got there

I connected to the Task 7 lab machine and ran ip address show eth1 followed by nmap -sn 192.168.12.0/24, finding alice (192.168.12.10) and bob (192.168.12.20). A follow-up nmap -sV scan showed bob with port 80 open (a Radicale CalDAV service) and alice with port 4444 open, flagged by nmap's own fingerprinting as returning a fixed whoami\npwd\n response regardless of probe type, an early hint it wasn't a normal service.

I tried curl http://192.168.12.20/ directly and got no auth header received, confirming I could reach the port but not its content without credentials. A passive tcpdump -A -i eth1 port 80 for 20-30 seconds captured nothing, since no traffic crosses a properly functioning switch to a port not addressed to me.

I then relaunched the ARP spoofing attack from Task 6 (ettercap -T -i eth1 -M arp) in one session while running tcpdump -A -i eth1 port 80 in a second. This time, since alice and bob run Ubuntu's default (unprotected) ARP handling, the poisoning succeeded and I captured alice's periodic HTTP requests to bob, including a GET /test.txt with an Authorization: Basic header. Decoding that base64 string (YWRtaW46czNjcjN0X1A0eno=) gave the credential admin:s3cr3t_P4zz. Using it directly (curl -u admin:s3cr3t_P4zz http://192.168.12.20/user.txt) returned the flag THM{wh0s_$n!ff1ng_0ur_cr3ds}.

Stopping ettercap with q printed its shutdown sequence, showing it explicitly "RE-ARPing the victims" to restore the genuine ARP associations before exiting, the proper way to leave a MITM position without leaving hosts with permanently poisoned ARP caches.

Separately, I restarted the ARP spoofing attack and ran tcpdump -A -i eth1 port 4444 for a few minutes, capturing repeated connections from bob into alice's port 4444 running the same three-command sequence each time (whoami → root, pwd → /root, ls → rev.go, root.txt, server.sh, www).

3.6 Man in the middle - Manipulation

What this section covered

This section extended the ARP spoofing MITM from passive sniffing to active packet manipulation. Rather than just reading traffic passing through eve, I used ettercap's -F filter option to rewrite packets in transit in real time, specifically, intercepting Alice's whoami command to Bob and substituting a different command entirely. This demonstrated that once an attacker has a MITM position, they aren't limited to eavesdropping, they can alter the conversation itself without either party's knowledge.

Question

What is the root.txt flag?

Answer

THM{wh4t_an_ev1l_M!tM_u_R}

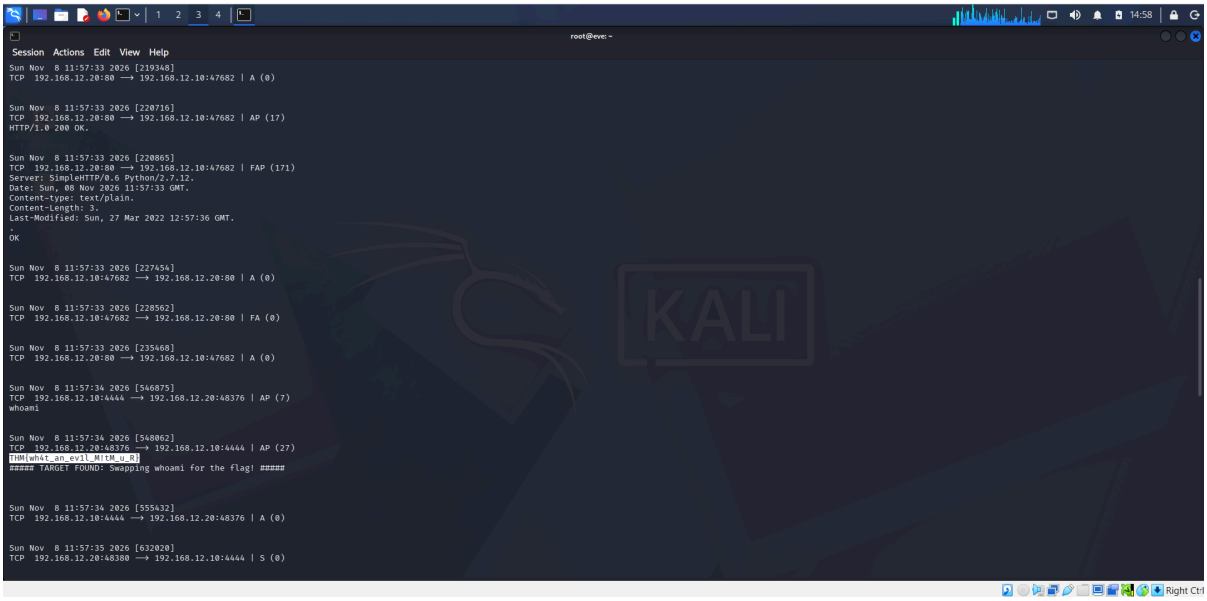
How you got there

I wrote an etterfilter source file (whoami.ecf) to match TCP traffic from Alice's source port 4444 containing the string "whoami", and replace it with cat root.txt\n so Bob would execute that instead: I compiled it with etterfilter whoami.ecf -o whoami.ef and loaded it into the ARP spoofing attack with ettercap -T -i eth1 -M arp -F whoami.ef.

I hit a few debugging steps along the way: my first .ecf draft failed to compile with a "wrong number of arguments for search" error, which turned out to be an artifact of how nano had saved the file rather than the syntax itself, rewriting it with a clean heredoc (cat > file << 'EOF' ... EOF) resolved it without changing the logic. I also briefly matched on tcp.dst == 4444 instead of tcp.src == 4444, which silently let traffic through unmodified since Alice's whoami packet is sourced from port 4444, not sent to it, correcting the field fixed it immediately. I initially built a full reverse-shell version of this filter (redirecting a bash shell back to a netcat listener on eve) and confirmed it worked, the connection landed successfully but lost it by pressing Ctrl+C on the live session before interacting with it; I switched to the simpler cat root.txt substitution instead, which returns the flag directly in the existing conversation without needing a second channel at all.

Evidence

Shows The ettercap's live output showing the intercepted whoami packet, my filter's "TARGET FOUND" message firing immediately after, and Bob's response containing the flag instead of "root".



Confirms successful in-flight packet manipulation: Bob executed a command he never actually received from Alice, proving the attacker fully controlled the content of the conversation, not just its visibility.

4. Challenges and how you resolved them

The main friction in this room was debugging the etterfilter syntax in Task 8, the compiler's "wrong number of arguments" error turned out to be unrelated to the actual logic (a file-saving quirk from nano, fixed by rewriting the file with a clean heredoc), and I initially matched the wrong TCP field (tcp.dst instead of tcp.src), which let traffic through silently with no error to flag the mistake. Both were resolved by comparing my filter's behavior against the real captured traffic rather than assuming the syntax was the only thing that could be wrong. I also briefly lost a working reverse shell by hitting Ctrl+C on the live connection before interacting with it, a reminder that once a shell lands, the next keystroke matters more than finishing the command that got you there.

5. Conclusion

Starting this TryHackMe room I could only define MAC flooding, ARP spoofing, MITM. Now I can trace them as one continuous chain: exhaust a switch's CAM table and it fails open into hub-like broadcasting; poison a host's ARP cache and it sends traffic to the wrong MAC entirely. Both abuse the fact that switches and the ARP protocol were designed to trust local traffic by default, with no verification built in.

What stood out most is that both attacks have well-established mitigations, and both belong on the switch, not the endpoints which is a genuinely useful distinction for a SOC analyst to carry forward, since it tells you where to point a hardening recommendation rather than just flagging "weak security" vaguely. MAC flooding is defeated by port security: configuring a switch port to allow only a limited, known number of MAC addresses (or a static allow-list), so a flood of spoofed addresses gets the port shut down or the excess traffic dropped rather than exhausting the CAM table. ARP spoofing is defeated by Dynamic

ARP Inspection (DAI), paired with DHCP snooping to build a trusted binding table of which MAC address legitimately owns which IP the switch then validates every ARP packet against that table and drops anything that doesn't match, which is exactly the kind of "ARP packet validation" Task 6 referenced when the hardened hosts rejected my poisoning attempt. Task 7 and 8 succeeded specifically because those protections were absent at the host level and nothing equivalent was enforced upstream on the switch either.

This connects directly to SOC work: it's not enough to recognize these attacks in traffic knowing that the fix is a switch configuration change (port security, DAI, DHCP snooping) rather than an endpoint fix (antivirus, patching) is what turns a detection into an actionable recommendation a network team can actually implement.